MOVIE TICKET BOOKING SYSTEM

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

CSA0801 – PROGRAMMING IN PYTHON

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

S. AARATHANA (1921260021)

Under the Supervision of

DR. SAMSON EBENEZAR

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Movie Ticket Booking System" has been carried out by S. Aarathana (1921260021) and Ayesha Siddiqua A (1911260141) under the supervision of Dr. Samson Ebenezer and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE FACULTY

Name: Dr.U.Samson Ebenezar

Designation: Professor

Department: CSE

COURSE COORDINATOR

Name: Dr.S.Loganayaki

Designation: Professor

Department: CSE

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

DECLARATION

We, S. Aarathana and Ayesha Siddiqua A of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Movie Ticket Booking System” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:Chennai

Date:07/09/26

Signature of the Students with Names

S. Aarathana:

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
S. Aarathana 1921260021	Movie management and staff-side functions	Movie display, add, search, update and delete operations; staff portal support	Tested movie records, price updates, searches and staff operations	Chapters 1–4; implementation and testing content	50%
Ayesha Siddiqua A 1911260141	Booking, snacks, billing and customer-side functions	Ticket booking, cancellation, seat/theatre/screen selection, snack billing, ratings and customer portal support	Tested booking, cancellation, bills, ratings and customer operations	Chapters 1–5; results, conclusion and report integration	50%
Total					100%

ABSTRACT

Movie ticket booking involves multiple activities such as maintaining movie information, searching available movies, booking and cancelling tickets, selecting theatre and screen details, ordering snacks, calculating bills, and reviewing booking information. Handling these operations manually can increase repetitive work and the possibility of data-entry errors. The Movie Ticket Booking System is developed as a menu-driven Python application integrated with a MySQL database to provide a structured solution for these operations. The system separates administrative functions through a Theatre Staff Portal and customer functions through a User Portal. Staff can display, add, search, update and delete movie records, view bookings, count bookings, identify most-watched movies, view ratings, and inspect theatre and seat information. Customers can search movies, select theatre, screen, language and seat tier, book or cancel tickets, submit ratings and reviews, order snacks, and generate snack and combined bills. Python functions organize the application logic, while mysql.connector enables communication with MySQL and SQL queries perform persistent data operations. Exception handling is used for invalid numeric inputs, duplicate identifiers and database-related errors. Testing of the implemented functions showed successful database connectivity, movie management, ticket booking and cancellation, snack ordering, billing, ratings and reporting. The project demonstrates a practical integration of Python programming and relational database operations in a console-based application. The current version does not maintain real-time seat availability, uses basic authentication, and does not include online payment. These limitations provide clear directions for future improvement, including real-time seat management, secure role-based authentication, online payment and automated booking confirmation.

Keywords: Python, MySQL, Movie Ticket Booking, Database Connectivity, Ticket Booking, Billing

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit	Chapter No.
DBMS	Database Management System	—	1, 3, 4
SQL	Structured Query Language	—	1, 2, 4
UI	User Interface	—	1, 3, 4
ID	Identifier	—	3, 4
VIP	Very Important Person	—	3, 4
IMAX	Image Maximum / large-format cinema system	—	3, 4

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

The Movie Ticket Booking System is a menu-driven application developed using Python with MySQL as the persistent data store. The application manages movie information and ticket booking data while also supporting snack orders, billing, ratings and reporting. The supplied project implementation uses Python functions for separate operations and mysql.connector for communication between the application and the database.

1.2 Need for the Project

Movie booking and theatre administration require consistent handling of movie records, ticket transactions, prices, customer selections and billing. When these activities are maintained manually or through disconnected records, repeated data entry and calculation can increase errors and administrative effort. A database-backed program provides a single structured mechanism for storing and retrieving the required records.

- Customers need a simple method to view movies, select booking details, cancel bookings, order snacks and obtain bills.
- Theatre staff need controlled access to movie management, booking records and reports.
- Persistent MySQL storage reduces dependence on temporary in-memory data.
- Automatic calculation of ticket and snack totals reduces manual calculation.

1.3 Problem Statement

To design and implement a Python–MySQL based Movie Ticket Booking System that manages movie records, customer bookings, cancellations, theatre and seat selections, snack orders, ratings, billing and booking-related reports through separate staff and customer portals, while handling invalid inputs and database errors in a structured manner.

1.4 Project Aim

To develop an efficient console-based Movie Ticket Booking System using Python and MySQL for managing movie information, ticket bookings, snack orders, billing, ratings and reports.

1.5 Project Objectives

1. To manage movie details using a MySQL database.
2. To search, add, update and delete movie records.
3. To book and cancel movie tickets through a customer portal.
4. To manage snack orders along with movie bookings.
5. To calculate movie, snack and combined bills.
6. To identify the most-watched and top movies based on booking records.
7. To demonstrate Python–MySQL connectivity and database operations.
8. To provide separate staff and customer portals for role-specific operations.

1.6 Scope

Included: movie display and management, movie search by ID/name/language/price, ticket booking and cancellation, theatre/screen/language/seat-tier selection, snack ordering and billing, movie ratings and reviews, booking counts, most-watched reports, and separate staff/customer portals.

Excluded: real-time database-driven seat availability, online payment, email/SMS confirmation, advanced customer authentication, movie posters/trailers, and advanced revenue analytics.

Assumptions and boundaries: theatre, screen and language choices are predefined in the Python program; the system runs as a console application; a working MySQL server and the required database tables are available.

1.7 Expected Engineering Outcome

The expected outcome is a working software prototype that connects Python to MySQL and performs role-based movie management and customer booking operations. The prototype should store transaction data persistently, calculate charges automatically, retrieve reports from stored records, and demonstrate correct handling of normal and invalid inputs.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review for this report is based on the references supplied with the project presentation. The cited material includes a survey on modern online ticket booking systems, a movie online ticket booking project, a case study on an online ticket booking system, a comparative study of online movie ticket booking, and a conceptual framework for multiplex theatre ticket booking. These sources were used as background references for identifying common functions expected in ticket-booking applications.

2.2 Review of Existing Work

The supplied references indicate continuing academic and project interest in online and computerized ticket booking systems. At a functional level, existing ticket-booking solutions generally focus on replacing manual reservation activities with structured digital processes. The present project adopts this general direction but implements the solution as a Python console application with MySQL persistence and combines movie management, booking, snacks, billing, ratings and simple reporting in one program.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual / register-based booking	Simple to understand; minimal software dependency	Repetitive work, manual calculations, difficult searching and reporting	Motivates database-backed automation
Basic computerized booking	Faster data entry and retrieval	May focus only on tickets and omit staff reports, snacks or ratings	Provides a baseline for feature integration
Proposed Python–MySQL system	Persistent records; separate portals; booking, snacks, ratings and reports in one application	Console-based; basic authentication; no real-time seat database or online payment	Selected implementation for the course project

2.4 Research / Engineering Gap

For this course-level project, the practical gap addressed is the need for a compact system that demonstrates multiple connected theatre operations within one Python–MySQL application. The project combines staff-side movie administration with customer-side booking, snack billing, ratings and simple analytical reports. The remaining gaps in the current prototype are real-time seat persistence, stronger authentication, online payment and automated notifications.

2.5 Proposed Contribution

The proposed contribution is an integrated console-based prototype with separate Staff and Customer portals. It connects Python functions with SQL operations and extends basic booking with theatre/screen/language/seat-tier selection, snack ordering, combined billing, movie ratings and booking-based reports.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Customer	View/search movies, book/cancel tickets, select theatre/screen/language/seat tier, order snacks, view bills and submit ratings.
Theatre Staff	Manage movie records, view bookings, update prices, delete eligible movies, view reports and ratings.
System / Database	Store movie, booking, snack-order and rating information and support reliable SQL operations.

Functional & Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
Movie management	Functional	Add, display, search, update and delete movie records
Ticket booking	Functional	Store valid booking details and calculate total amount
Snack management	Functional	Order snacks and calculate snack/combined bill
Ratings & reports	Functional	Store ratings and generate booking-related summaries
Portal separation	Functional	Separate staff and customer menus
Input handling	Non-Functional	Reject or handle invalid numeric and database inputs
Persistence	Non-Functional	Commit valid operations to MySQL
Maintainability	Non-Functional	Organize operations into reusable Python functions

3.2 Constraints & Applicable Standards

The supplied project files do not specify a formal external engineering standard. Therefore, this report does not claim compliance with an unverified IEEE/ISO standard. The applicable project constraints are derived from the implemented design.

Constraint / Code	Requirement	How Applied in This Project
Technical constraint	Console-based Python program with MySQL connectivity	Design uses Python functions, mysql.connector and SQL queries.
Database integrity	Booking and movie identifiers must be handled consistently	Duplicate IDs and database errors are handled through exceptions; commits occur after valid operations.
Security constraint	Staff access uses predefined passwords; customer login is basic	Documented as a limitation and security risk rather than claimed as secure authentication.
Operational constraint	Theatre/screen/language data are predefined	Selections are provided from Python constants rather than dedicated database tables.

3.3 Design Specifications

Parameter	Target/Requirement	Unit / Value	Priority
Database connection	Connect to MySQL database successfully	Connection state	High

Movie search	Support ID, name and language search	3 search modes	High
Staff portal	Provide administrative menu	15 menu choices	High
Customer portal	Provide customer menu	14 menu choices	High
Rating range	Accept valid movie rating	1.0–5.0	Medium
Seat tiers	Support Silver, Gold and Platinum/VIP	3 tiers	Medium
Billing	Calculate movie, snack and combined totals	Automatic	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Manual/register-based booking. Records and calculations are maintained manually. It is simple but weak for searching, persistence, automated billing and reporting.

Alternative 2: Python-only application using temporary in-memory structures. It can automate calculations but data is lost when the program ends unless separate file handling is added.

Alternative 3: Python with MySQL. Application logic is handled in Python while persistent records and queries are handled by MySQL. This matches the implemented project.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	25%	2	3	4
Cost	15%	4	4	4
Safety / Data consistency	15%	1	2	3
Sustainability / reduced paper dependence	10%	1	3	4
Reliability / persistence	35%	1	1	4

3.5 Selected Design & Detailed Design

Alternative 3, Python with MySQL, was selected because it provides persistent storage while retaining straightforward Python-based application logic. It directly supports the course objective of demonstrating Python–MySQL connectivity and database operations.

Figure 3.1 – System Architecture / Workflow

Login	Portal Selection	Movie Operations	Booking	Snacks / Ratings	Billing / Reports	MySQL Database

Systems approach: The login layer directs the user to either the Staff Portal or Customer Portal. Portal menu functions call specific Python functions for movie, booking, snack, rating and report operations. These functions execute SQL statements through a shared MySQL cursor. Successful data-changing operations are committed to the database, while selected exceptions are handled and reported to the user.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Persistent storage	Python-only temporary storage	Records remain available between sessions	Requires MySQL setup and connection	Use MySQL because persistence is essential to the project.
Separate portals	Single common menu	Clear separation of staff and customer operations	More menu logic	Use separate portals for role-specific functionality.

Predefined theatre/screen data	Dedicated database tables	Simple implementation	Not dynamically maintainable	Use predefined values for current prototype; database tables are future work.
Console UI	Graphical/web interface	Low complexity and suitable for course implementation	Less user-friendly than GUI/web	Use console interface for current scope.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reduce dependence on manual paper-based booking records	Database stores movie and booking records digitally	Database-backed record keeping selected.
Ethics	Maintain accurate records and acknowledge external resources	Project format requires acknowledgement; database transactions are explicitly stored	References and limitations are disclosed; fabricated results are avoided.
Health & Safety	Avoid misleading claims about physical safety features	Project is software-only and has no direct physical control function	No irrelevant safety standard is claimed.
Security	Restrict staff functions and protect database operations	Staff login exists, but passwords are predefined in source code	Separate staff login retained; weak password design recorded as a limitation and future improvement.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Database connection failure	Medium	High	High	Check connection status; catch connector errors	Medium
Duplicate booking/movie ID	Medium	Medium	Medium	Catch integrity errors and request unique IDs	Low
Invalid numeric input	Medium	Low	Low	Use ValueError handling	Low
Weak staff password storage	Medium	High	High	Restrict current prototype; recommend secure authentication in future	Medium
Incorrect/duplicate seat selection	Medium	Medium	Medium	User selects seat text; future database-driven seat validation required	Medium
Data loss from failed transaction	Low	High	Medium	Commit only successful operations; rollback in delete error path	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project followed a modular development approach. Requirements for movie management, booking, snacks, ratings and billing were first identified, after which separate Staff and Customer portals were designed. Python functions were implemented for individual operations and connected to MySQL using mysql.connector. The modules were then integrated into the main menu and tested using valid, invalid and database-related input cases.

Resource	Purpose
Python source program	Application logic and menu-driven operations
MySQL database	Persistent movie, booking, snack-order and rating data
mysql.connector	Python–MySQL connectivity
Test input records	Verification of searches, bookings, billing, cancellation and reports

4.2 Software Implementation & Modern Tools Used

The application begins by connecting to the MySQL database and creating the movie_ratings table if it does not already exist. Predefined dictionaries and lists store theatre, screen and language options. The program is organized into functions such as show_movies(), book_ticket(), add_movie(), search_movie(), show_bookings(), update_price(), delete_movie(), order_snacks(), full_bill(), rate_movie(), staff_login(), user_login(), staff_portal() and user_portal(). SQL SELECT, INSERT, UPDATE and DELETE statements are executed through the cursor, and data-changing operations are committed to MySQL.

Tool / Software / Equipment	Purpose	Stage Used
Python	Core application logic, functions, menus and input handling	Development
MySQL	Persistent relational data storage	Database design and execution
mysql.connector	Connect Python application to MySQL	Integration
SQL queries	CRUD operations, joins, aggregates and reports	Implementation and reporting
Exception handling	Handle invalid numeric input, duplicate IDs and connector errors	Testing and reliability

Implementation evidence to attach in the final submitted copy: screenshots of the main login menu, staff portal, customer portal, movie list, booking confirmation, snack bill/combined bill and ratings output.

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Database connection	Run application with configured MySQL server	Connection successful and menus start
Staff login	Enter recognized staff/admin username and valid predefined password	Staff portal opens
Invalid staff login	Enter invalid staff credentials	Access denied
Movie search	Search by valid ID/name/language	Matching movie record(s) displayed
Ticket booking	Choose valid movie and enter booking details	Booking stored and confirmation total displayed
Duplicate booking ID	Reuse existing booking ID	Integrity error handled
Cancel booking	Enter existing booking ID	Booking removed and success message displayed
Snack order	Enter valid booking, snack and quantity	Snack order stored and cost calculated

Combined bill	Enter valid booking ID	Movie cost + snack cost = grand total
Rating	Enter rating within 1.0–5.0	Rating/review stored
Invalid rating	Enter value outside 1.0–5.0	Rating rejected
Exit	Select Exit Program	Database cursor/connection closed

Verification Summary

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Python–MySQL connectivity	Successful connection	Successful connection	Pass
Movie management	CRUD/search operations work	Implemented in staff portal	Pass
Booking	Store and cancel booking	Implemented	Pass
Snack billing	Calculate snack total	Implemented	Pass
Combined billing	Movie + snack total	Implemented	Pass
Ratings	Store 1–5 rating and review	Implemented	Pass
Real-time seat availability	Database-maintained seat state	Not implemented	Not Achieved

4.4 Results

The implemented system successfully connected Python with MySQL and provided the required menu-driven operations. Movie records could be displayed and managed, ticket bookings could be stored and cancelled, snack orders and bills could be calculated, ratings and reviews could be stored, and booking-based reports such as most-watched and top movies could be generated. Separate Staff and Customer portals were successfully integrated.

Function Group	Observed Result	Validation
Movie management	Display, add, search, update and delete functions available	Successful
Ticket booking	Booking stored with calculated total	Successful
Cancellation	Existing booking can be removed	Successful
Snack operations	Order, bill and total snack amount supported	Successful
Combined bill	Movie and snack costs combined	Successful
Ratings and reviews	Stored and displayed with average/recent review output	Successful
Reports	Booking counts, most-watched and top-three movie reports	Successful

4.5 Data Analysis & Engineering Judgment

The results show that a modular Python program can coordinate multiple database-backed theatre operations without requiring a complex graphical interface. SQL aggregation is used for booking counts, most-watched movies, top-three movies, snack totals and average ratings, reducing manual calculation. The main engineering limitation is that seat numbers are entered as text and are not checked against a persistent seat-availability table. Similarly, authentication is intentionally basic. These limitations mean that the prototype demonstrates functional integration but should not be treated as a production-ready commercial booking platform.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Manage movie details	Movie CRUD/search functions connected to MySQL	Fully Achieved
Book and cancel tickets	book_ticket() and cancel_booking() implemented	Fully Achieved
Manage snacks	Snack menu, ordering and billing functions	Fully Achieved
Calculate combined bill	full_bill() adds movie and snack cost	Fully Achieved
Generate booking reports	Booking count, most-watched and top-three functions	Fully Achieved
Demonstrate Python–MySQL connectivity	mysql.connector connection and SQL operations	Fully Achieved

Provide production-grade seat/payment workflow	No real-time seat table or online payment	Not Achieved / Outside Current Scope

4.7 Strengths & Limitations

Strengths:

- Clear separation between Staff and Customer portals.
- Persistent MySQL storage for core records.
- Modular Python functions for major operations.
- Automatic ticket, snack and combined bill calculation.
- Movie ratings/reviews and simple analytical reports.
- Exception handling for common invalid inputs and database errors.

Limitations:

- Real-time seat availability is not maintained in the database.
- Staff passwords are predefined in the Python source code.
- Customer authentication is basic.
- The system is console-based.
- Online payment integration is not implemented.
- Theatre and screen information is predefined in Python.

4.8 Project Planning, Roles & Budget

Milestone Timeline

Stage	Milestone	Work	Status
1	Requirement analysis	Identify booking, movie, snacks, ratings and billing requirements	Completed
2	System and database design	Plan portals, functions and MySQL data operations	Completed
3	Module development	Implement movie, booking, snack, rating and report functions	Completed
4	Integration	Connect modules through main menu and portals	Completed
5	Testing	Verify valid/invalid inputs and database operations	Completed
6	Documentation	Prepare presentation and capstone report	Completed

Roles and Contribution

Student	Assigned Responsibility	Actual Contribution
S. Aarathana	Movie management and staff-side functions	Movie CRUD/search, staff operations, related testing and documentation
Ayesha Siddiqua A	Booking, customer portal, snacks, billing and ratings	Booking/cancellation, customer operations, billing/ratings testing and report integration

Budget

Item	Estimated Cost	Actual Cost
Python	₹0	₹0
MySQL	₹0	₹0
Development on existing computer	₹0 additional project software cost	₹0 additional project software cost
Total additional software cost	₹0	₹0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Movie Ticket Booking System was developed as a Python–MySQL based console application to simplify movie management, ticket booking and cancellation, snack ordering, billing, ratings and booking-related reports. The system successfully integrates separate Staff and Customer portals and uses SQL operations for persistent storage and retrieval. Testing confirmed the major implemented functions, including movie management, booking, cancellation, snack billing, combined billing and ratings. The project therefore demonstrates the practical use of Python functions, exception handling, SQL queries and database connectivity in an integrated software prototype. The system meets its intended course-level objectives while clearly identifying limitations such as basic authentication, predefined theatre data, lack of real-time seat persistence and absence of online payment.

5.2 Future Work

- Add real-time seat availability and seat locking.
- Add online payment integration.
- Add email/SMS booking confirmation.
- Add secure administrator and customer role management.
- Create dedicated theatre, screen and seat database tables.
- Add movie posters and trailers through a graphical or web interface.
- Add advanced revenue and sales reports.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Python–MySQL connectivity using mysql.connector.
- Use of SQL CRUD operations, joins and aggregate queries from Python.

- Design of separate role-based menu flows.
- Exception handling for database and user-input errors.
- Integration of booking, snacks, billing and ratings into one application.

Engineering & Professional Skills Developed

- Requirement analysis and modular decomposition.
- Database-backed software design.
- Testing with valid and invalid inputs.
- Team division of responsibilities and report preparation.
- Technical documentation and presentation of results.

Individual Reflection – S. Aarathana

The project improved my understanding of how Python functions can be connected with MySQL to manage structured records. My major contribution focused on movie management and staff-side operations, where I worked with display, search, update and delete functions. Testing these operations helped me understand the importance of input validation and database consistency. If the project were repeated, I would improve the staff authentication and move more predefined information into database tables.

Individual Reflection – Ayesha Siddiqua A

The project helped me understand how separate Python modules can work together in a complete database application. My major contribution focused on customer booking, cancellation, snack billing, ratings and combined billing. I learned how SQL queries and Python calculations can be integrated to produce stored transactions and useful outputs. If the project were redesigned, I would add real-time seat availability, stronger customer authentication and a more user-friendly graphical interface.

REFERENCES

[1] A. N. Mhetre, M. Kadam, and S. Kalambe, “A Survey on Modern Online Ticket Booking Systems,” International Journal of Scientific Research in Computer Science, Engineering and Information Technology, vol. 10, no. 6, pp. 981–987, 2024. DOI: 10.32628/CSEIT2410460.

[2] O. Thakre, A. Pawar, S. Raut, S. Kharge, and A. Shaik, “A Project on Booking Movie Online Ticket System,” International Journal of Innovative Science and Research Technology, vol. 9, issue 3, 2024. DOI: 10.38124/ijisrt/24mar1247.

[3] K. Acharya, “A Case Study on Online Ticket Booking System Project,” SSRN, 2024. DOI: 10.2139/ssrn.4841210.

[4] A. Roy, V. Shahdeo, and R. Kaluri, “A Comparative Study in Online Movie Ticket Booking System,” 2021.

[5] B. Harshitha, P. B. S. Jyothi, K. L. Phanindra, T. P. Venkat, and B. T. S. S. Srihari Krishna Gopal, “Movie Ticket Booking System for Multiplex Theatre: A Conceptual Framework for Design, Architecture, and User Experience.”

[6] Python source code developed for the Movie Ticket Booking System capstone project, 2026.

[7] MySQL database and mysql.connector module used for project implementation, 2026.

APPENDICES

Appendix A – Detailed Calculations

Ticket booking calculation: price_per_ticket = base_price + tier_extra. The program assigns tier_extra as Rs. 0 for Silver, Rs. 50 for Gold, and Rs. 100 for Platinum/VIP. The final ticket amount is calculated as total = price_per_ticket × number_of_tickets. This amount is stored in the booking table together with the booking ID, movie ID, ticket count and booking date.

Snack calculation: after a valid Snack ID is selected, the snack price is fetched from the snacks table. The program calculates total snack cost as total = snack_price × quantity and stores the result in snacks_order. For a booking, the total snack amount is obtained using SUM(total_cost).

Combined bill calculation: the movie booking amount is fetched from booking and the snack amount is obtained from snacks_order. If no snack order exists, snack cost is treated as zero. Grand Total = Movie Cost + Snacks Cost. Rating analysis uses AVG(rating) and COUNT(rating_id), while booking reports use COUNT(booking.movie_id) and ordering to identify the most-watched and top three movies.

Appendix B – Engineering Drawings / Schematics

System workflow used in the implemented program:
Main Program → Login Selection → [Theatre Staff Login / User Login] → Role-specific Portal → Functional Module → SQL Query → MySQL Database → Result/Confirmation.

Staff flow: Staff Login → Show/Add/Search Movies → View Bookings → Update/Delete Movie → Price Search → Booking Reports → Seat/Theatre Details → Ratings → Snack Report → Logout.

Customer flow: User Login → Show/Search Movies → Seat/Theatre/Screen Details → Ticket Booking → Cancellation → Rating/Review → Snack Order/Bill → Combined Bill → Logout.

During ticket booking, the customer selects Movie ID, theatre, screen, language, seat tier, ticket count and seat number(s). The program calculates the amount, inserts the booking into MySQL and displays a booking confirmation.

Appendix C – Source Code / Code Repository Information

The complete project source is a Python program connected to MySQL through mysql.connector. The code is organized into functional modules: database connection and initialization; movie display and management; ticket booking and cancellation; theatre, screen, language and seat details; snack ordering and billing; movie ratings and reviews; booking reports; staff login and portal; customer login and portal; and the main program loop. The complete repository/link was not supplied in the project files, so no repository URL is invented here.

Essential implementation excerpts and function inventory:

Database and rating-table initialization:

```
mydb = mycon.connect(host="127.0.0.1", port=3306, user="root", database="movie", use_pure=True, connection_timeout=5)
cur = mydb.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS movie_ratings (rating_id INT AUTO_INCREMENT PRIMARY KEY, movie_id INT, username VARCHAR(50), rating FLOAT, review VARCHAR(255),
```

rating_date DATE)")

Ticket amount logic:
price_per_ticket = base_price + tier_extra
total = price_per_ticket * t

Major functions: show_movies(), book_ticket(), add_movie(), search_movie(), show_bookings(), update_price(), delete_movie(), search_by_price(), count_movie_bookings(), most_watched_movie(), top_3_movies(), cancel_booking(), show_snacks(), order_snacks(), show_snack_bill(), total_snack_amount(), most_ordered_snack(), full_bill(), show_seat_details(), show_theatres_and_screens(), rate_movie(), show_movie_ratings(), staff_login(), user_login(), staff_portal(), and user_portal().

Appendix D – Datasheets

No hardware datasheet is required because the submitted implementation is software-based. The relevant software/interface specifications are: Python for application logic; MySQL for persistent relational storage; mysql.connector for database connectivity; datetime.date for recording booking/rating dates; and a console/terminal for user interaction. The program expects a local MySQL connection on host 127.0.0.1, port 3306, using the database named movie. Authentication secrets are intentionally not reproduced in this report.

Appendix E – Experimental / Raw Data

Test evidence available from the project implementation and presentation is summarized below.
T01 – Database connection: program connects to MySQL and creates movie_ratings when required – Result: Successful.
T02 – Movie management: display, add, search, update and delete operations – Result: Successful.
T03 – Ticket booking: valid Movie ID, ticket count and booking details are stored and total amount is calculated – Result: Successful.
T04 – Booking cancellation: an existing Booking ID can be deleted – Result: Successful.
T05 – Snack ordering: snack price is fetched, quantity is accepted and total snack cost is calculated – Result: Successful.
T06 – Combined billing: movie amount and snack amount are added to display the grand total – Result: Successful.
T07 – Ratings and reviews: rating from 1.0 to 5.0 and review text are stored in movie_ratings – Result: Successful.
T08 – Reports: booking count, most-watched movie, top three movies and most-ordered snack are generated using aggregate SQL queries – Result: Successful.
T09 – Invalid input handling: invalid numeric input and selected database errors are handled through exceptions/messages – Result: Implemented.
T10 – Real-time seat availability: persistent seat locking/availability is not maintained – Result: Not implemented; recorded as a limitation.

Appendix F – Ethics / Institutional Approval

No separate institutional ethics approval document was supplied and the project does not involve human-subject experimentation, medical intervention or physical fabrication. Ethical considerations relevant to this software project are accurate storage of booking/rating data, truthful reporting of results, acknowledgement

of references and disclosure of limitations. Security-related limitations are also reported: staff passwords are predefined in the source code and customer authentication is basic. A production version should use secure credential storage, role-based access control and stronger validation.

Appendix G – Project Meeting / Progress Records

The supplied project materials do not contain dated meeting minutes. Therefore, the progress record is documented phase-wise without inventing dates:

Phase 1 – Requirement Analysis: identified movie booking, movie management, snacks, ratings, billing and reporting requirements.

Phase 2 – System Design: planned separate Staff and Customer portals and menu flow.

Phase 3 – Database Design: organized movie, booking, snacks/snack-order and rating data in MySQL.

Phase 4 – Python Development: implemented functions, user input, SQL queries and exception handling.

Phase 5 – Integration: connected Python and MySQL and combined all modules under the main login/menu system.

Phase 6 – Testing: checked valid/invalid inputs, bookings, cancellations, billing, ratings and database operations.

Phase 7 – Documentation: prepared project presentation and final capstone report.

Appendix H – User/Industry Feedback

No formal user/industry feedback form or external evaluation record was included in the supplied project files. This appendix is therefore retained for completeness. If faculty, theatre users or classmates provide documented feedback later, the signed/dated feedback sheet or summarized comments can be inserted here without changing the technical results already reported.

Appendix I – Publications / Patents / Competitions / Product Outcomes

No publication, patent, competition certificate or product-commercialization outcome was supplied for this project. Therefore, there is no project outcome to report under this appendix at present. The section is retained because it is part of the prescribed capstone report format.

Appendix J – Individual Contribution Evidence

Contribution evidence is aligned with the 50%–50% division stated in the report.

S. Aarathana (1921260021) – 50%: movie management and staff-side functions; contribution includes movie display, add, search, update and delete operations, staff portal support, testing of movie records/price/search operations, and contribution to implementation documentation.

Ayesha Siddiqua A (1911260141) – 50%: customer booking and transaction functions; contribution includes ticket booking and cancellation, theatre/screen/language/seat selection, snack ordering and billing, ratings/reviews, customer portal support, testing of booking/billing/rating operations, results/conclusion content and report integration.

Evidence available in the submitted project materials includes the joint project presentation naming both students and the integrated Python source containing the corresponding modules. Any additional screenshots, version history or signed work log can be attached here if required by the assessor.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)